WHITE PAPER

Change Data Capture from SAP Business Data Cloud

A reference architecture for incremental capture from catalog-linked shares

Incremental capture from zero-copy shares in Snowflake, without copying the share and without building an engine

Audience	Data architects, platform engineers and data leaders at organisations consuming SAP Business Data Cloud data products in Snowflake
What you will be able to do	Determine whether change data capture is viable on your own share, then deploy and operate it using three SQL files and one configuration change
Prerequisites	SAP BDC data products linked into a Snowflake account; a role that can see them and create objects in a Gold database
Reference implementation	github.com/dfreriks-snow/sap-bdc-cdc (Apache 2.0)
Evidence base	58 measurements against a live SAP BDC share over a five-day observation window
Published	19 August 2026

Contents

1 Abstract
2 How SAP BDC data reaches Snowflake
3 The problem: a zero-copy share carries no change signal
4 What Snowflake permits on a catalog-linked share
5 The precondition: does the connector merge, or reload?
6 Why the replication watermark is the right predicate
7 Reference architecture
8 Implementation, step by step
Step 1 Confirm what you have
Step 2 Run the readiness assessment
Step 3 Deploy the control schema
Step 4 Deploy monitoring and capture a baseline
Step 5 Configure the capture engine
Step 6 Verify what Snowflake actually chose
Step 7 Tune the refresh interval
Step 8 Add the exception path only if you need it
9 Operating it
10 What capture does not solve
11 Limits, open questions and how to verify locally
12 Conclusion
A Capability reference
B Troubleshooting by symptom

HOW TO USE THIS PAPER

Sections 1 to 7 establish whether the pattern applies to you and why it works. Section 8 is the implementation and can be followed on its own once you have run the assessment in Step 2. Sections 10 and 11 state plainly what this does not fix and which questions remain open, because a reference architecture that only lists its strengths is not usable for planning.

1 Abstract

SAP Business Data Cloud publishes data products into Snowflake as catalog-linked databases: read-only, zero-copy, with no storage cost and no pipeline to operate. The appeal is obvious. The difficulty is less so - a share that copies nothing also carries nothing that tells a consumer what changed since they last looked. The common consequence is a Gold layer that re-reads the entire share on a schedule, which is correct but expensive twice: once in compute, and once in latency, because the schedule has to be loosened until the cost becomes tolerable.

This paper shows that change data capture is achievable on such a share, that the mechanism is an existing Snowflake platform feature rather than something to be built, and how to deploy and operate it. Three findings carry the argument.

- **The three mechanisms most teams try first are unavailable.** Standard streams, the CHANGES clause and change tracking are all refused, because the shared objects are Iceberg tables on an external catalog rather than ordinary tables - notwithstanding that they report otherwise.
- **The BDC connector merges rather than reloads.** Historical rows keep their original replication timestamp while new data lands in new files. This is the precondition for any incremental design, it is a property of the customer's replication pipeline rather than a platform guarantee, and it can be verified in a minute.
- **Dynamic tables refresh incrementally over the share.** Verified across the query shapes a real Gold layer uses. A refresh with no source change returns NO_DATA in a few hundred milliseconds, meaning Snowflake establishes that nothing changed without recomputing the source. That is the capture engine, and it requires no code.

A replication watermark is available alongside it for the two cases dynamic tables cannot serve: a transformation containing a non-deterministic expression, and a target outside Snowflake. On the tenant measured for this paper, a watermark predicate returned the changed rows while scanning zero megabytes of a seven-gigabyte object; the business-date predicate most teams reach for first scanned 4.7 GB for the same answer.

THE DISTINCTION THAT MATTERS MOST FOR PLANNING

Two complaints usually arrive together from consumers of a BDC share, and only one is solvable here. Latency - the data is hours or a day behind - is a platform problem, is within your control, and is the subject of this paper. Completeness - a required measure or dimension is absent from the share - is a data product problem that no refresh frequency addresses, because the fields are missing rather than stale. Delivering fifteen-minute freshness on a dataset that cannot answer the question being asked is a convincing answer to the wrong question.

2 How SAP BDC data reaches Snowflake

SAP BDC Connect for Snowflake publishes each data product as its own catalog-linked database. Snowflake reads the underlying files in place through an external Iceberg catalog; nothing is copied and the customer pays no storage for it. Three characteristics follow, and all three shape what is possible.

Characteristic	Consequence
One read-only database per data product	Object counts grow with the number of products subscribed. A consolidation layer above them is worth establishing early, while the count is small
Objects are Iceberg tables on an external catalog	They report kind = TABLE and is_external = N, which understates what they are. This single fact determines which change mechanisms exist - see section 4
INFORMATION_SCHEMA.COLUMNS returns no rows	The column list can only be obtained with DESCRIBE TABLE, one statement per object. Any generated transformation layer must cache the result rather than resolve it at build time
Schema names contain a colon; object names are mixed case	Every reference must be double-quoted, for example MY_PRODUCT_DB."my_product_srv:v1"."i_someobject". Unquoted SQL and much self-service tooling will fail against it

SAP BDC pipelines built on SLT commonly add a customer extension column carrying the replication write time. The name varies by implementation - ZZLASTCHGDATE is the most common spelling encountered - and its presence is what makes the watermark path in this paper possible. Its absence is the one condition under which the pattern reduces to dynamic tables only. Section 8, Step 2 detects it either way.

3 The problem: a zero-copy share carries no change signal

Nothing about a catalog-linked share forces a full re-read. It is simply what you get by default when no change signal is available and none is sought.

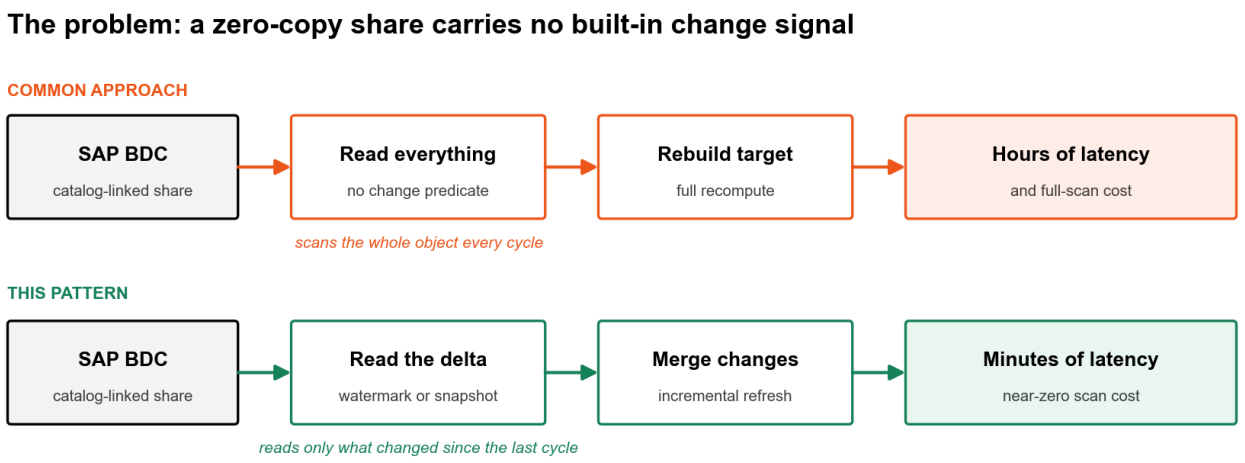


Figure 1. The common approach, and the alternative this paper describes. The difference is not effort; it is whether a change signal is identified and used.

The complaint that reaches the data team is almost always about latency, and the instinct is to tighten the schedule. That treats the symptom. The cause is that every cycle does the same amount of work regardless of how little changed, so the schedule is the only lever and it is a lever with a cost attached to every notch.

4 What Snowflake permits on a catalog-linked share

Because the objects are Iceberg tables on an external catalog, the usual change-capture primitives behave differently from how they would on a native table. Snowflake's own refusals are the clearest statement of what is available, so they are quoted verbatim rather than paraphrased. Every row below was established by attempting the operation against a live share.

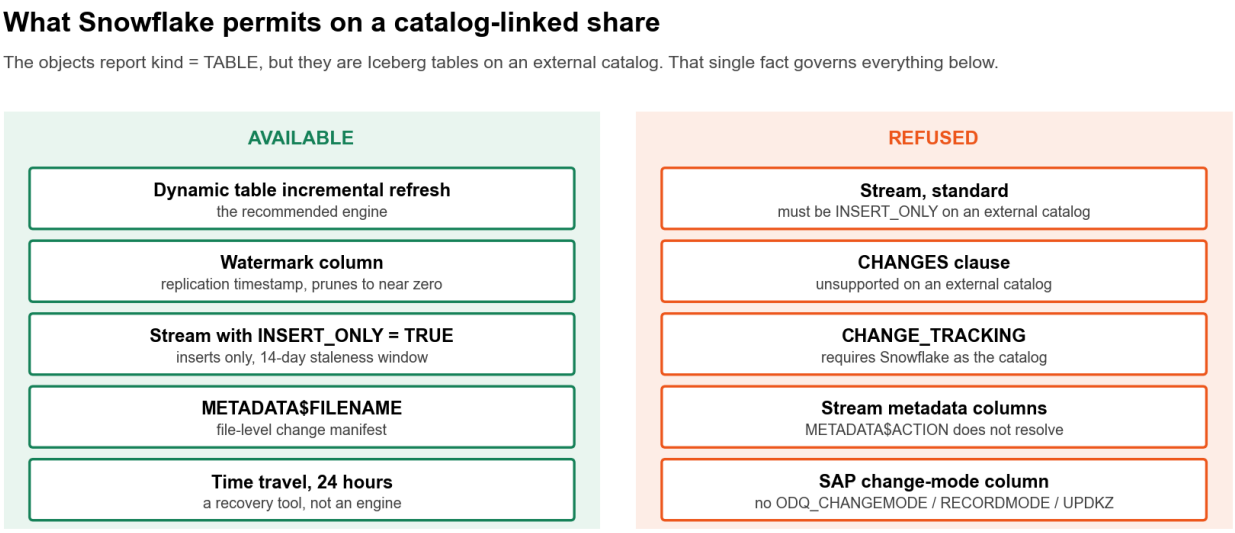


Figure 2. Available and refused mechanisms. The single cause of the right-hand column is the external catalog.

Mechanism	Status	Evidence, verbatim where a refusal
Dynamic table, REFRESH_MODE = INCREMENTAL	**Available**	Accepted with refresh_mode_reason = null. The recommended engine
Replication watermark column	**Available**	A customer extension column carrying the write time; prunes to near zero (section 6)
Stream with INSERT_ONLY = TRUE	**Available**	Created successfully. type = DELTA, mode = INSERT_ONLY, 14-day staleness window
METADATA$FILENAME	**Available**	Resolves and exposes the underlying Parquet path, enabling file-level change detection
Time travel, AT and BEFORE	**24 hours**	Available at 24 h, unavailable at 36 h. Retention is the account default of one day and the share cannot be altered to extend it
Stream, standard	**Refused**	"Streams on External Tables or Iceberg tables with an external catalog must have INSERT_ONLY set to true."
CHANGES clause	**Refused**	"CHANGES clause is not supported on external tables or Iceberg tables with an external catalog."
CHANGE_TRACKING	**Refused**	"change_tracking cannot be set for Iceberg tables unless they use Snowflake catalog." The share is read-only in any case
METADATA$ACTION, $ISUPDATE, $ROW_ID	**Unavailable**	Do not resolve on the base object, since change tracking is not active
SAP-side change-mode column	**Absent**	No ODQ_CHANGEMODE, RECORDMODE or UPDKZ anywhere in the share

4.1 The consequence that matters most

The absence of a change-mode column is the most important limitation in the table above, and it falls asymmetrically on the two viable mechanisms. A dynamic table detects a deleted row because it compares source snapshots - it does not need to be told by a flag. A watermark-driven MERGE cannot: a row the share stops carrying simply stops appearing, and nothing in the remaining data records that it went.

This is the single strongest argument for preferring dynamic tables, and the reason any watermark implementation must reconcile row counts against the source on a schedule and treat divergence as a signal to

rebuild.

SAP business deletion flags do exist on some objects - a deletion indicator on the supplier master, marked-for-deletion on the product and the GL account, a deletion code on purchasing documents. These are useful and must be filtered in the consumed layer, but they describe a business state. They are not a replication record mode and they will not tell a capture process that a row has been removed from the share.

5 The precondition: does the connector merge, or reload?

Everything downstream rests on one behaviour. If the connector rewrote the whole share on each cycle, every row would carry the latest write timestamp and nothing could separate changed data from unchanged. Observed over five days on the reference tenant, it does not: historical rows keep their original stamp and new data arrives in new files.

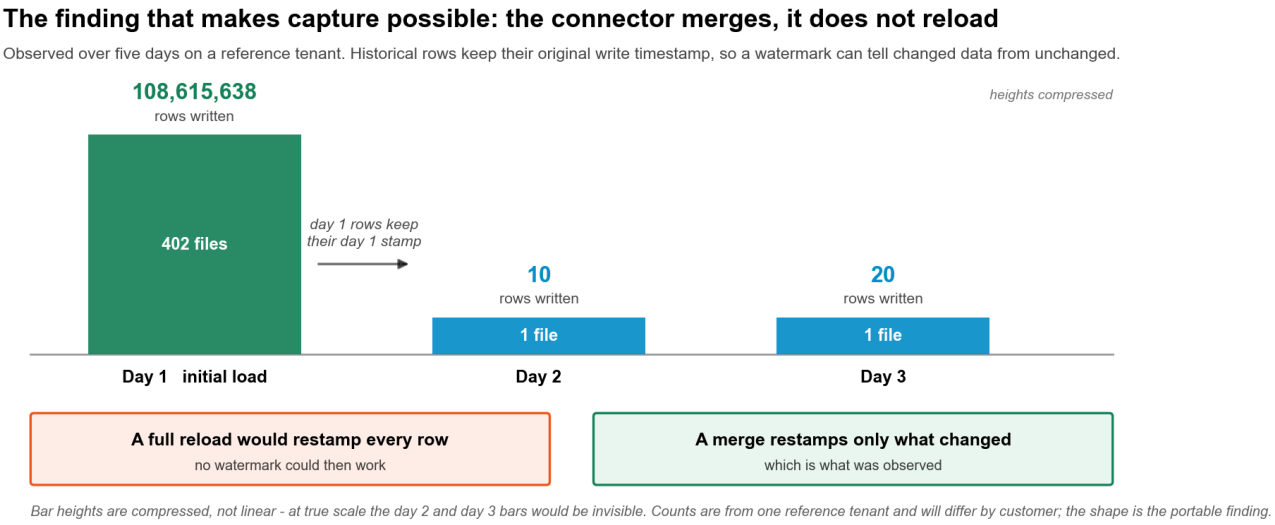


Figure 3. Reference-tenant observation. The portable finding is the shape - one bulk load, then small deltas in new files - not the row counts.

VERIFY THIS ON YOUR OWN TENANT; DO NOT INHERIT IT FROM THIS PAPER

This is a property of a customer-managed replication pipeline. It is not guaranteed by Snowflake, it is not documented as a contract, and it could change. The assessment in Step 2 tests it directly, and the monitor in Step 4 exists to catch it changing later. A share with only one write day so far cannot be distinguished from one that reloads - in that case, re-run after the next refresh.

5.1 Late writes include retroactive updates

An append-only design would be simpler and cheaper, and for some objects it is sufficient. For others it is not. On the reference tenant, sales order lines created more than a year earlier were rewritten during a later day's delta - which is entirely normal, since orders are repriced, rescheduled and partially delivered. The implication is that a capture process cannot assume a late write is a new record. It must MERGE on the business key rather than INSERT. Getting this wrong does not error; it silently duplicates rows.

6 Why the replication watermark is the right predicate

Once the connector is known to merge, the replication timestamp becomes usable as a watermark. What makes it a good one is not that it exists, but where it sits relative to physical file layout.

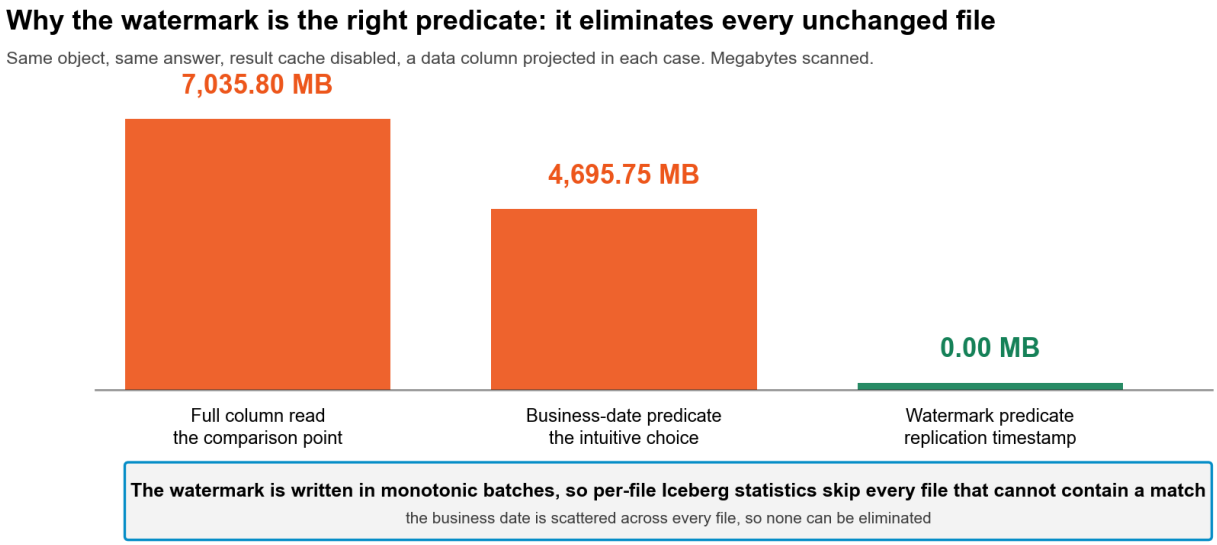


Figure 4. Reference tenant, result cache disabled, a data column projected in each case. Megabytes scanned to return the same answer.

The connector writes the timestamp in monotonic batches, so each underlying Parquet file spans a narrow and largely non-overlapping range of it. Iceberg holds per-file minimum and maximum statistics for every column, so Snowflake eliminates every file that cannot contain a matching row. Where only a handful of rows changed, that is almost every file.

A business date has no such correlation with layout. Records from every period are scattered across every file, so nothing can be eliminated and the predicate degenerates into a full scan that then discards nearly everything it read. This is worth naming explicitly because filtering on a business date is the intuitive first attempt, and it costs about as much as not filtering at all.

6.1 The same effect applies to whole transformations

This is what determines whether an hourly Gold layer is affordable. On the reference tenant a wide GROUP BY aggregate over 108.6 million rows scanned 6.87 GB and took 15.1 seconds; the identical aggregate scoped by the watermark scanned nothing measurable and took 630 milliseconds. A no-change dynamic table refresh of the same shape returned in 450 milliseconds without recomputing anything.

A PRACTICAL IMPLICATION FOR COST ESTIMATES

An aggregate that looks too expensive to refresh hourly usually is not, once it only processes what changed. Estimates built on full-rebuild assumptions can be wrong by orders of magnitude, in the direction that causes teams to abandon a viable design.

THIS IS BEHAVIOUR, NOT A CONTRACT

The pruning follows from how the connector currently writes. If the underlying files were ever compacted, rows carrying old timestamps would be rewritten into new files, per-file ranges would widen, and pruning would degrade toward a full scan. Nothing would error. This is why Step 4 monitors file count alongside the reload test.

7 Reference architecture

Incremental capture adds no layer. If you already run a medallion architecture over the share, the raw and consolidation layers are unchanged and remain zero-copy; what changes is how the consumed layer is configured, plus one small additive control schema beside it.

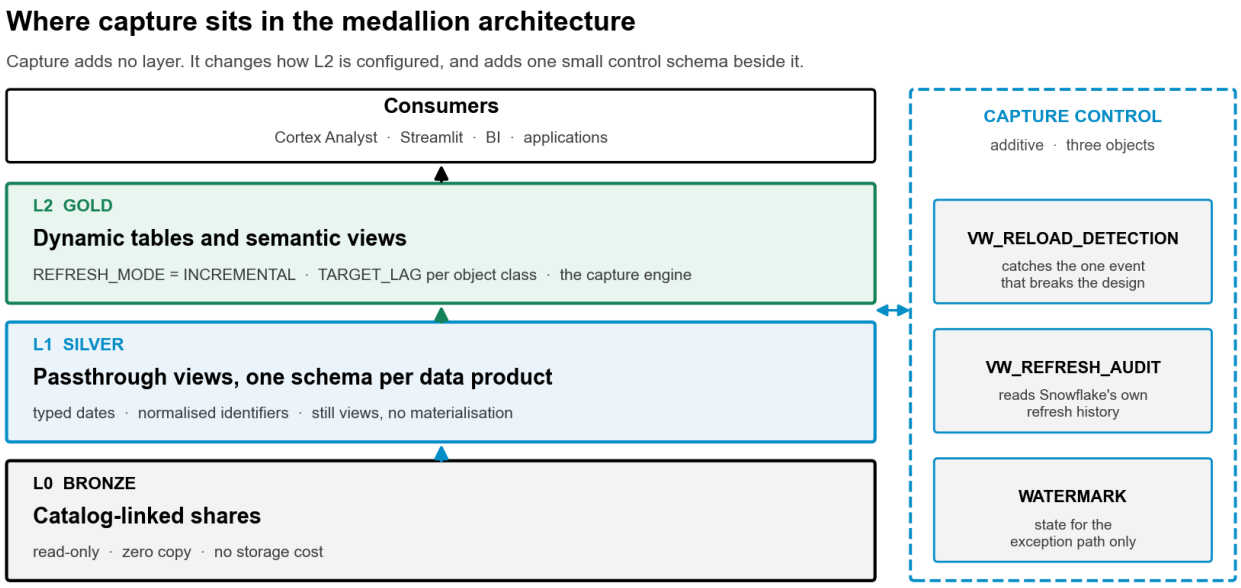


Figure 5. Bronze and Silver unchanged; Gold changes configuration; a three-object control schema is added.

Layer	Contents	What capture changes
Bronze	The catalog-linked shares themselves. Read-only, zero-copy, no storage cost	Nothing. The change signal is read from here; nothing is written
Silver	One passthrough view per shared object, typically casting date strings and normalising identifiers. Still views - no materialisation	Nothing. Verified not to block incremental refresh, including the type casts
Gold	Materialised business entities and semantic objects	REFRESH_MODE stated explicitly as INCREMENTAL; TARGET_LAG set per object class. This layer becomes the capture engine
Control	New: watermark state, refresh audit, baseline	Additive. Three objects, and the reload monitor is the one to deploy first

If you do not have a Silver layer, the pattern still works - dynamic tables can read the catalog-linked objects directly. A consolidation layer is worth having for other reasons (quoting, type casting, one schema per product), but it is not a prerequisite for capture.

7.1 Choosing a mechanism

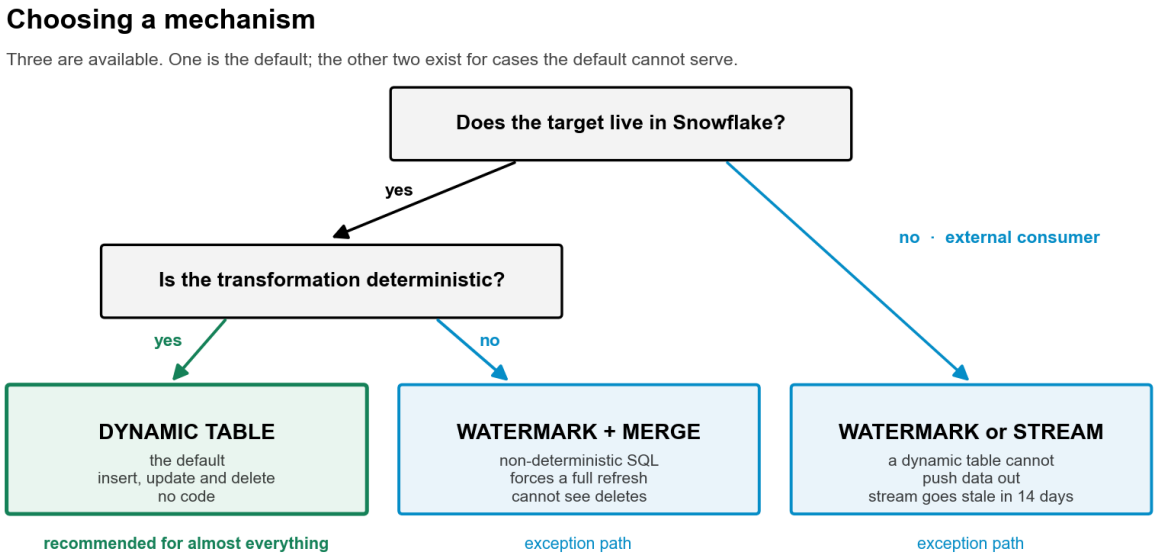


Figure 6. Dynamic tables for anything landing inside Snowflake. The other two paths exist for specific cases and are weaker in one important respect.

	Dynamic table	Watermark + MERGE	INSERT_ONLY stream
Inserts and updates	yes	yes	yes
Deletes	yes - diffs snapshots	no - no change-mode column	no - by definition
Cheap no-change detection	yes	yes	yes
Code to write and maintain	none	task, control table, MERGE per object	task, MERGE per object
Can feed a target outside Snowflake	no	yes	yes
Goes stale if unread	no	no	yes, 14 days
Blocked by non-deterministic SQL	yes	no	no

Use dynamic tables. They are the only mechanism that captures deletes and the only one that needs no code. The insert-only stream is listed for completeness rather than recommended: it provides nothing the other two lack, and it carries a fourteen-day staleness window after which it must be recreated and the gap backfilled.

8 Implementation, step by step

The order matters. Monitoring is deployed before the engine is configured, so that a baseline exists to compare against - without one, a share that has always behaved a certain way is indistinguishable from one that changed last night. Placeholders in angle brackets are yours to substitute.

Step	Deliverable	Effort
1 Confirm what you have	An inventory of linked databases and the watermark column	Minutes, read-only
2 Run the readiness assessment	A verdict with evidence	Minutes, read-only
3 Deploy the control schema	Three objects	One SQL file
4 Deploy monitoring, capture a baseline	Reload detection, exceptions view, alert	One SQL file
5 Configure the capture engine	Dynamic tables with explicit mode and per-class lag	One configuration change per object
6 Verify what Snowflake chose	Confirmation that refresh is genuinely incremental	Two queries
7 Tune the refresh interval	A lag matched to observed connector cadence	After a week of observation
8 Add the exception path	Only for non-deterministic SQL or an external target	Optional, per object

1 Confirm what you have

Find the linked databases by kind, never by name, and locate the watermark.

Naming conventions vary between tenants. The database kind that BDC Connect sets does not, which makes it the portable way to discover the share.

```
SHOW DATABASES;
-- keep the rows where "kind" = 'CATALOG-LINKED DATABASE'

SHOW TABLES IN DATABASE <LINKED_DB>;
SHOW ICEBERG TABLES IN DATABASE <LINKED_DB>;
-- the same objects are returned by both: they report kind = TABLE but are
-- Iceberg tables on an external catalog, which is what governs section 4

DESCRIBE TABLE <LINKED_DB>."<schema:v1>"."<object>";
-- the only route to a column list; INFORMATION_SCHEMA.COLUMNS returns no rows
```

In the DESCRIBE output, look for a TIMESTAMP column whose name suggests load or replication rather than a business event - ZZLASTCHGDATE is the most common. A business date such as a posting or document date is **not** a watermark, and section 6 explains why using one as though it were is the most expensive mistake available here.

2 Run the readiness assessment

One command answers whether the pattern applies to your tenant, with evidence.

```
git clone https://github.com/dfreriks-snow/sap-bdc-cdc
cd sap-bdc-cdc/probes
pip install -r requirements.txt

# read-only
python3 cdc_readiness.py --connection <your_connection> --report readiness.md

# also creates a stream and a dynamic table in a scratch database and drops them,
# which proves the recommended mechanism rather than inferring it
python3 cdc_readiness.py --connection <your_connection> --with-scratch-tests \
--report readiness.md --json readiness.json
```

The assessment discovers the share by kind, finds the watermark on each object, tests merge-versus-reload, measures pruning, and optionally proves the mechanisms. Each finding carries the measurement behind it in the JSON output.

Verdict	Means	Your next move
READY	A dynamic table refreshes incrementally and detects no-change cheaply	Continue to Step 3
READY WITH CAVEATS	Either you ran read-only so the mechanism was not proven, or a check was inconclusive - most often the merge test on a share with only one write day so far	Re-run with --with-scratch-tests. If the merge test is the open item, re-run after the next connector refresh
NOT READY	No watermark and no incremental dynamic table; there is no change signal	Raise the load-timestamp ask in section 10 with your pipeline owner before proceeding
NOT ASSESSABLE	No catalog-linked database is visible to the current role	Check grants before concluding nothing has been shared

TWO MEASUREMENT TRAPS, IF YOU MEASURE BY HAND INSTEAD

First, disable the result cache for the duration of any cost comparison - the original pruning measurement for this paper was cache-contaminated and reported an excellent result for the wrong reason. Second, project a real column and confirm the full-scan baseline is non-zero: COUNT(*) and COUNT(DISTINCT col) can both be answered from Iceberg manifest statistics without reading data, and on a single-valued column even SUM(LENGTH(col)) scans nothing. A zero baseline means change the probe column, not that pruning is perfect. A third: query history needs a database context, and it must be a real database - a catalog-linked database has no usable INFORMATION_SCHEMA, so reading history from inside one silently returns nothing.

3 Deploy the control schema

Three objects: watermark state, refresh audit, baseline. From sql/01.

```
CREATE SCHEMA IF NOT EXISTS <GOLD_DB>.CAPTURE_CONTROL;

CREATE OR REPLACE VIEW <GOLD_DB>.CAPTURE_CONTROL.VW_REFRESH_AUDIT AS
SELECT NAME, SCHEMA_NAME, STATE, REFRESH_ACTION, REFRESH_TRIGGER,
REFRESH_START_TIME, REFRESH_END_TIME,
DATEDIFF('millisecond', REFRESH_START_TIME, REFRESH_END_TIME) AS DURATION_MS,
STATISTICS:numInsertedRows::NUMBER AS ROWS_INSERTED,
STATISTICS:numDeletedRows::NUMBER AS ROWS_DELETED,
STATISTICS:numCopiedRows::NUMBER AS ROWS_COPIED,
IFF(REFRESH_ACTION = 'FULL' AND REFRESH_TRIGGER <> 'CREATION', TRUE, FALSE)
AS UNEXPECTED_FULL_REFRESH
FROM TABLE(<GOLD_DB>.INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY());
```

ONE DETAIL THAT WILL OTHERWISE COST YOU AN HOUR

STATISTICS is returned as an OBJECT, not a VARCHAR. Address it with the colon path operator as above. Wrapping it in TRY_PARSE_JSON is the natural thing to write and fails with "Invalid argument types for function 'TRY_PARSE_JSON': (OBJECT)". This was found by compiling the statement rather than by reading it.

4 Deploy monitoring and capture a baseline

The one thing to deploy even if you deploy nothing else. From sql/03.

Two events would silently make incremental capture as expensive as a full rebuild, and neither raises an error: the connector switching to full reload, and the underlying files being compacted. A third - the connector stalling - leaves every refresh reporting success while the data quietly ages. All three are detected by measurement or not at all.

```
CREATE OR REPLACE VIEW <GOLD_DB>.CAPTURE_CONTROL.VW_RELOAD_DETECTION AS
WITH per_object AS (
SELECT '<OBJECT_LABEL>' AS OBJECT_NAME,
COUNT(*) AS TOTAL_ROWS,
SUM(IFF(<WATERMARK> >= DATEADD('hour', -24, CURRENT_TIMESTAMP()), 1, 0))
AS ROWS_STAMPED_24H,
COUNT(DISTINCT METADATA$FILENAME) AS FILE_COUNT,
MAX(<WATERMARK>) AS LAST_WRITE
FROM <LINKED_DB>."<schema:v1>"."<object>"
-- UNION ALL one branch per monitored object
)
SELECT OBJECT_NAME, TOTAL_ROWS, ROWS_STAMPED_24H, FILE_COUNT, LAST_WRITE,
ROUND(100.0 * ROWS_STAMPED_24H / NULLIF(TOTAL_ROWS,0), 6) AS PCT_RESTAMPED,
CASE WHEN ROWS_STAMPED_24H > 0.50 * TOTAL_ROWS THEN 'FULL RELOAD DETECTED'
WHEN DATEDIFF('hour', LAST_WRITE, CURRENT_TIMESTAMP()) > 24
THEN 'CONNECTOR STALLED'
ELSE 'INCREMENTAL - HEALTHY' END AS CAPTURE_MODE
FROM per_object;

-- capture the baseline immediately; drift is only meaningful against a start point
INSERT INTO <GOLD_DB>.CAPTURE_CONTROL.CAPTURE_BASELINE
(OBJECT_NAME, TOTAL_ROWS, FILE_COUNT, LAST_WRITE, PCT_RESTAMPED_24H)
SELECT OBJECT_NAME, TOTAL_ROWS, FILE_COUNT, LAST_WRITE, PCT_RESTAMPED
FROM <GOLD_DB>.CAPTURE_CONTROL.VW_RELOAD_DETECTION;
```

The fifty per cent threshold is deliberately far from normal operation. On the reference tenant, thirty rows out of 108.6 million were restamped over two days - 0.00003 per cent - so half cannot fire by accident. Monitoring the largest one or two objects per data product is sufficient, because the connector's mode is a property of the

pipeline rather than of an individual table.

5 Configure the capture engine

Two choices carry all the weight, and both are easy to get wrong by omission.

5.1 State the refresh mode; never leave it as AUTO

AUTO lets Snowflake decide, and it falls back to FULL when it cannot support incremental for a given query. Nothing announces the difference - no error, no warning, just a larger bill and a longer lag. Stating INCREMENTAL forces Snowflake either to honour it or to refuse at CREATE time, which converts a silent cost regression into a deployment-time error.

There is no reason to leave it to AUTO out of caution. All four shapes a real consumed layer uses were verified to sustain incremental refresh over a catalog-linked share.

Query shape	Initial refresh	Second refresh, no source change
Flat projection over a view that casts types	INCREMENTAL	**NO_DATA in 348 ms**
Join of two views	INCREMENTAL	**NO_DATA in 486 ms**
Aggregate with GROUP BY	INCREMENTAL	**NO_DATA in 450 ms**
Left join to a pre-aggregated child CTE	INCREMENTAL	**NO_DATA in 570 ms**

NO_DATA is the result that matters: Snowflake established that nothing had changed without recomputing the source. That is exactly the behaviour a capture engine needs, and it is why no engine has to be written.

```
CREATE OR REPLACE DYNAMIC TABLE <GOLD_DB>.<SCHEMA>.DT_SALES_ORDER_LINE
TARGET_LAG = '15 minutes'
WAREHOUSE = <GOLD_WH>
REFRESH_MODE = INCREMENTAL -- stated, never AUTO
INITIALIZE = ON_CREATE
AS
SELECT i.SALESDOCUMENT, i.SALESDOCUMENTITEM, i.MATERIAL, i.PLANT,
i.NETAMOUNT, i.ORDERQUANTITY, i.ORDERQUANTITYUNIT,
h.SALESORGANIZATION, h.SOLDTOPARTY, h.SALESDOCUMENTDATE,
i.<WATERMARK> AS SOURCE_WRITE_TS -- carry it through for lineage and freshness
FROM <SILVER_DB>.<SCHEMA>.SALES_ORDER_ITEM i
JOIN <SILVER_DB>.<SCHEMA>.SALES_ORDER_HEADER h
ON h.SALESDOCUMENT = i.SALESDOCUMENT;
```

5.2 Set the lag per object class, not uniformly

A no-change refresh costs a few hundred milliseconds, so a uniform lag is not expensive - but it is uninformed. Transactional objects change continuously; master data dimensions may not have been written for days. On the reference tenant, six of twenty-three objects had not been written in five days and carried a single distinct write timestamp, while the transactional objects were written the same afternoon. The assessment output includes the per-object watermark and distinct-stamp counts needed to classify them.

Object class	How to identify it from the assessment	Suggested TARGET_LAG
Transactional, high change	Written on most or all days in the observation window	15 minutes
Transactional, low change	Written during the initial load, rarely since	1 hour
Master data, active	Occasional writes; more than a handful of distinct stamps	1 hour
Master data, static	One distinct write stamp; nothing since the initial load	24 hours, or DOWNSTREAM

5.3 What not to put inside a dynamic table

Any non-deterministic expression forces a full refresh on every cycle and defeats the whole pattern. CURRENT_DATE(), CURRENT_TIMESTAMP() and RANDOM() are the usual culprits, and month-to-date logic is the common way one gets introduced by accident. Put it in a view over the dynamic table: the view is always current, costs nothing, and leaves the table incrementally refreshable.

```
-- wrong: forces a full refresh every cycle
CREATE DYNAMIC TABLE ... AS
SELECT ... WHERE SALESDOCUMENTDATE >= DATE_TRUNC('month', CURRENT_DATE());

-- right: the dynamic table stays incremental, the view stays current
CREATE OR REPLACE VIEW <GOLD_DB>.<SCHEMA>.VW_MONTH_TO_DATE AS
SELECT * FROM <GOLD_DB>.<SCHEMA>.DT_SALES_ORDER_LINE
WHERE SALESDOCUMENTDATE >= DATE_TRUNC('month', CURRENT_DATE());
```

6 Verify what Snowflake actually chose

Do not trust the request; read the outcome. Two queries.

```
SHOW DYNAMIC TABLES IN SCHEMA <GOLD_DB>.<SCHEMA>;
-- refresh_mode should be INCREMENTAL and refresh_mode_reason should be NULL

SELECT NAME, REFRESH_ACTION, REFRESH_TRIGGER, DURATION_MS, ROWS_INSERTED
FROM <GOLD_DB>.CAPTURE_CONTROL.VW_REFRESH_AUDIT
ORDER BY REFRESH_START_TIME DESC LIMIT 20;
```

What you see	What it means
NO_DATA, a few hundred milliseconds	Healthy. Nothing changed, and Snowflake established that without recomputing
INCREMENTAL, small ROWS_INSERTED	Healthy. Changed rows only
INCREMENTAL, ROWS_INSERTED equal to the full row count	Expected on the first refresh only. On any later refresh, investigate
FULL in steady state	A problem. Either the mode was left as AUTO and downgraded, or a non-deterministic expression has been introduced. This is the condition to alert on

7 Tune the refresh interval

From observed connector cadence, not from ambition.

Once no-change refreshes are confirmed to be sub-second, the lag can be tightened toward what consumers actually asked for. Observe the connector for a week first: a target lag materially shorter than the interval at which the connector writes buys nothing and pays for the privilege. Set the lag to roughly the connector's own write interval, and revisit if that interval changes.

8 Add the exception path only if you need it

Two cases justify it. Everything else should be a dynamic table.

The watermark path is warranted where a transformation contains a non-deterministic expression that cannot be lifted into a view, and where the target sits outside Snowflake, since a dynamic table cannot push data anywhere. It is strictly weaker in one respect - it cannot detect physical deletes - so it comes with a mandatory reconciliation.

```
SELECT MAX(<WATERMARK>) INTO v_to FROM <source>; -- fix the ceiling BEFORE reading

MERGE INTO <target> AS t
USING (SELECT ... FROM <source>
WHERE <WATERMARK> > :v_from
AND <WATERMARK> <= :v_to) AS s -- closed at both ends
ON t.<key1> = s.<key1> AND t.<key2> = s.<key2>
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT (...) VALUES (...);
```

- **Fix the high-water mark before reading, not from the rows afterwards.** A row written while the run is in progress is then picked up next cycle rather than skipped. Deriving the mark from MAX() of what you just read guarantees the opposite.
- **Close the range at both ends.** A failed run is then safe to repeat because it consumes the same rows. An open upper bound consumes a different set on each attempt, which makes a retry non-deterministic.
- **Reconcile row counts on a schedule.** The watermark cannot see a row the share has stopped carrying. A persistent positive divergence against the source means rows were deleted upstream; rebuilding the target is usually cheaper and safer than trying to identify them.

MERGE KEYS CANNOT BE GUESSED

A catalog-linked table declares no primary key, so the grain must be supplied - and SAP keys are frequently wider than they appear. Purchase order history is keyed on seven parts, and the two most likely to be omitted are exactly the two that discriminate: the account assignment number separates a line whose cost is split across accounts, and the history category separates a goods receipt from an invoice receipt posted against the same material document. On a five-part key the object appears to contain duplicates, and de-duplicating on that key discards real goods movements. Take merge keys from a verified grain and record them in the control table rather than in a developer's head.

9 Operating it

The design deliberately has few alerts. Three conditions matter, and all three are silent - none raises an error, and two leave the data correct while quietly multiplying the cost. That is what makes them worth monitoring, rather than the usual pipeline failures, which do not exist here because there is no pipeline at the raw or consolidation layer to fail.

Condition	How it presents	Detection	Severity
Connector switches from merge to full reload	Every row restamped. Refresh stays correct but costs the same as a rebuild; the watermark path reprocesses everything	VW_RELOAD_DETECTION, PCT_RESTAMPED above 50	**High**
Underlying files are compacted	Per-file watermark ranges widen and pruning degrades from near-zero toward a full scan	FILE_COUNT falling sharply against the baseline	**High**

Condition	How it presents	Detection	Severity
A refresh silently does full work	No error. Cost rises and lag lengthens with nothing to point at	VW_REFRESH_AUDIT, UNEXPECTED_FULL_REFRESH	**High**
Connector stalls	Data quietly ages while every refresh reports success and NO_DATA	VW_RELOAD_DETECTION, staleness rather than error	**Medium**
Non-deterministic expression introduced	That one object reverts to full refresh on every cycle	Same audit view; the object never reports NO_DATA	**Medium**
Merge key wrong on the watermark path	Duplicate rows, or discarded records if de-duplicated on an incomplete key	Row count against distinct full-key count	**Medium**

Route all of these through a single exceptions view so that an alert has exactly one dependency, and alert on high severity only. The reference implementation does this in sql/03_monitoring.sql, on an hourly schedule.

10 What capture does not solve

Capture makes data current. It does not make it correct, and it cannot create fields that were never shared. If a ledger arrives without amount columns, or a sales share arrives without a customer dimension, no refresh frequency addresses that - and it is worth being explicit with stakeholders early, because the two complaints usually arrive together.

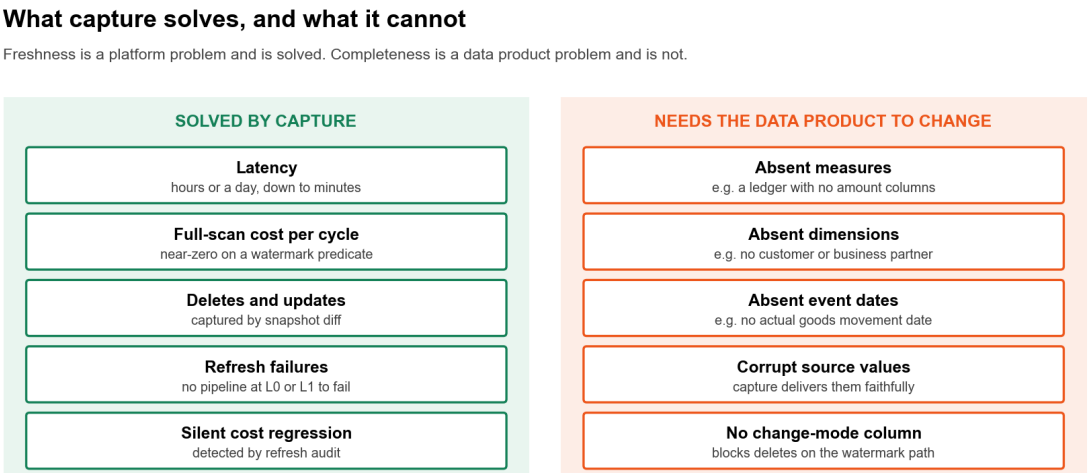


Figure 7. Freshness is a platform problem and is solved. Completeness is a data product problem and is not.

10.1 Requests worth making to the SAP side

Request	Why it matters	Consequence if absent
A load timestamp on every shared object	It is the watermark. Without one there is no change signal for the exception path	Dynamic tables become the only option; anything needing an external target is blocked
A change-mode or record-mode column	It is what makes a physical delete visible to a watermark-driven process	The watermark path cannot detect deletes and must reconcile row counts on a schedule
Documented grain and keys per object	A catalog-linked table declares no primary key, and some SAP keys are not guessable	Merge keys are inferred, and inference fails silently in both directions
Populate or remove empty contract columns	A column present in the schema but universally empty is discovered only after something depends on it	Design-time assumptions fail at test time or later

10.2 Requests worth making to Snowflake

Each of these follows directly from a refusal in section 4 rather than from a wish list. The pattern in this paper works, but it is inference from observed behaviour rather than a supported contract - and that is the underlying request.

Request	Current behaviour	What it would enable
CHANGES() on externally catalogued Iceberg	Refused: "not supported on external tables or Iceberg tables with an external catalog"	The most direct route to a change feed, without inferring anything from a replication timestamp
Change tracking without owning the catalog	Refused: "cannot be set for Iceberg tables unless they use Snowflake catalog"	Full stream semantics - insert, update and delete - rather than INSERT_ONLY
Longer or configurable time travel on linked shares	Retention is the account default of one day; the share cannot be altered	Snapshot diffing as a recovery mechanism beyond a single day
A first-class change feed for catalog-linked databases	None exists; this pattern reads a customer extension column and relies on connector write behaviour	A supported contract in place of a behavioural dependency that could change without notice

HOW TO RUN THE TWO CONVERSATIONS

Keep them separate and run them in parallel. Latency is within your control and can be delivered against the share exactly as it stands, with no dependency on SAP. Absent measures and dimensions are a data product change with a lead time outside your control. Conflating them stalls the deliverable that could ship this week behind the one that cannot.

11 Limits, open questions and how to verify locally

11.1 What generalises, and what does not

The mechanisms, the refusals and their causes are properties of Snowflake and of how BDC Connect publishes an Iceberg table on an external catalog. Those should hold on any tenant, and Step 2 confirms it in a minute.

The magnitudes do not generalise. Row counts, byte counts, file counts, object counts and the name of the watermark column are all specific to the tenant they were measured on. Where this paper quotes a figure it is to convey an order of magnitude - that a watermark predicate can eliminate essentially all reading, that a no-change refresh is sub-second - not to predict what you will observe.

One behavioural dependency deserves restating as a limit rather than a caveat: the pattern rests on the connector merging rather than reloading, and that is a property of a customer-managed replication pipeline. It is not guaranteed and it could change. That is precisely why the reload monitor is the first thing deployed and the last thing to remove.

11.2 Two questions this paper does not settle

Open question	What is established	Test to run on your tenant
What a refresh does when the source has genuinely changed	The initial refresh is INCREMENTAL, and a refresh with no source change returns NO_DATA in a few hundred milliseconds. A changed-source refresh was not observed in the window	Read VW_REFRESH_AUDIT after the next connector write. Expect INCREMENTAL with ROWS_INSERTED matching the changed source rows, not the full count
Whether an INSERT_ONLY stream yields rows on a source change	The stream was created successfully and correctly returned zero rows at creation, since it is positioned at the current state. It was not exercised against a change	Only relevant if you adopt the stream path, which is not recommended. Create it, wait for a write, then select from it

Neither changes the recommendation. If a changed-source refresh were to fall back to FULL, the fallback is the watermark path in Step 8, which is specified and whose cost is measured. The recommendation would move from "no code" to "a procedure per object" - not from workable to unworkable.

12 Conclusion

Change data capture from an SAP BDC catalog-linked share is achievable on the data exactly as shared, and the engine is a platform feature rather than a build. Dynamic tables configured with an explicit incremental refresh mode capture inserts, updates and deletes, and establish that nothing has changed in a few hundred milliseconds without recomputing the source. A replication watermark covers the two cases they cannot serve, and prunes so effectively that an incremental cycle over a multi-gigabyte object can read nothing measurable at all.

Four things are worth carrying away. State REFRESH_MODE explicitly, because AUTO permits a silent and expensive downgrade. Deploy the reload monitor before anything else, because the one assumption the pattern depends on is behavioural and its failure is silent. Prefer dynamic tables over a hand-built watermark engine, because only the former sees deletes. And separate the latency conversation from the completeness conversation, because the first is yours to fix this week and the second is not.

Steps 1 through 6 are configuration and three small objects. None of them depends on SAP resolving anything. That is the practical conclusion: unlike the data product gaps in section 10, incremental capture is entirely within your control and can be delivered against the share as it stands today.

> **VERIFY BEFORE YOU RELY ON IT**
> This is a community reference pattern, not a Snowflake product commitment. Everything in it was measured against a live SAP BDC share, and the assessment that produced those measurements is open source specifically so you can re-run it against your own account rather than trusting a figure from someone else's.

Appendix A Capability reference

A single-page summary for the reader who wants only the constraints. All statements verified against a live catalog-linked share.

Question	Answer
Can I create a standard stream on the share?	No. Snowflake requires INSERT_ONLY = TRUE on an Iceberg table with an external catalog
Can I create an INSERT_ONLY stream?	Yes. type = DELTA, mode = INSERT_ONLY, 14-day staleness window. Returns zero rows at creation, which is correct
Can I use the CHANGES clause?	No. Unsupported on an externally catalogued Iceberg table
Can I enable CHANGE_TRACKING?	No. Requires Snowflake as the catalog. The share is read-only in any case
Are METADATA$ACTION and friends available?	No. They do not resolve on the base object
Is METADATA$FILENAME available?	Yes. It exposes the underlying Parquet path and enables file-level change detection
How far back does time travel reach?	24 hours. Unavailable at 36. Retention is the account default of one day and cannot be raised on the share
Will a dynamic table refresh incrementally?	Yes, with REFRESH_MODE = INCREMENTAL accepted and no downgrade reason, across flat projections, joins, aggregates and left joins to pre-aggregated children
Is there an SAP-side change-mode column?	No. This is why a watermark cannot detect a physical delete and a dynamic table can
Does INFORMATION_SCHEMA.COLUMNS work?	No. It returns no rows. Use DESCRIBE TABLE, one statement per object
Can I read query history from inside a linked database?	No. It has no usable INFORMATION_SCHEMA; switch to a real database first
Is STATISTICS from refresh history a JSON string?	No, it is an OBJECT. Use the colon path operator; TRY_PARSE_JSON on it fails

Appendix B Troubleshooting by symptom

Symptom	Likely cause	What to do
Assessment reports NOT ASSESSABLE	The role cannot see the linked databases, or nothing has been shared	Check grants first. SHOW DATABASES as a role with broader visibility
Assessment cannot decide merge versus reload	The share has only one write day so far - a first load and a reload look identical	Re-run after the next connector refresh. Do not assume either way
Pruning check reports a zero-byte full scan	The probe column was answerable from Iceberg statistics; on a single-valued column even SUM(LENGTH(col)) reads nothing	Use a higher-cardinality column. The assessment now tries several automatically
Watermark predicate scans nearly as much as a full read	The column is not correlated with file layout, or the files have been compacted	Check FILE_COUNT against the baseline. Prefer dynamic tables, whose refresh does not depend on this
REFRESH_ACTION shows FULL in steady state	REFRESH_MODE was left as AUTO and downgraded, or a non-deterministic expression was introduced	Recreate with REFRESH_MODE = INCREMENTAL stated. If it is then refused, find and lift the non-deterministic expression into a view

Symptom	Likely cause	What to do
CREATE DYNAMIC TABLE refused with INCREMENTAL	The query cannot be incrementally maintained as written	Simplify the shape, or move the offending expression into a view above the table. Fall back to the watermark path only if neither works
Everything reports NO_DATA but the data is stale	The connector has stalled. Refreshes correctly report no change	VW_RELOAD_DETECTION will show CONNECTOR STALLED. This is a pipeline issue, not a Snowflake one
Duplicate rows appeared after adopting the watermark path	The merge key is narrower than the object's real grain	Compare row count against distinct full-key count. Take the grain from documentation, not inference
Rows exist downstream that the source no longer has	A physical delete, which the watermark path cannot see	Run the reconciliation query. Rebuilding the target is usually cheaper than identifying them
A query against the share fails on the object name	Schema names contain a colon and object names are mixed case	Double-quote every part: DB."schema:v1"."object"

About this paper. Every claim was measured against a live SAP Business Data Cloud catalog-linked share on a Snowflake account, over a five-day observation window, using an open assessment harness published alongside it. Figures attributed to a reference tenant are from a single account and will differ on any other; the mechanisms and their limits are what generalise. Two questions are recorded as open in section 11 rather than presented as settled. The reference implementation, including the assessment, the deployable SQL and the diagrams used here, is at github.com/dfreriks-snow/sap-bdc-cdc under Apache 2.0. This is a community reference pattern and not a Snowflake product commitment; verify on your own tenant before relying on it. Published 19 August 2026.